Celularni automati – Conway’s Game of Life
(Python i Rust implementacija)

Izveštaj o eksperimentima jakog i slabog skaliranja

Autor: Teodora Belić SV55/2022
Predmet: Napredne tehnike programiranja

1. Eksperimentalno okruženje

1.1 Tehnički detalji softverske I hardverske arhitekture sistema

Eksperimenti su izvršeni na sledećem sistemu:

- Model procesora: **Intel® Core™ Ultra 5 135U**
- Radni takt: **1.60GHz**
- Broj fizičkih jezgara: **12**
- Broj logičkih jezgara: **14**
- Organizacija cache memorije:
  - L1 cache: 1.2 MB
  - L2 cache: 10.0 MB
  - L3 cache: 12.0 MB
- Broj NUMA node-ova: 1
- Tip RAM memorije: DDR5
- Kapacitet RAM memorije: 32 GB
- Operativni sistem: Windows 11 Pro
  (Multiprocessing implementacija koristi **spawn** mehanizam za kreiranje procesa)
- Python verzija: 3.12
- NumPy: 2.2.6
- Matplotlib: 3.10.7
- multiprocessing modul (standardna biblioteka)
- Rust: 1.91.1
- Cargo build u release režimu

2. Metodologija merenja

Za svaku kombinaciju:

- broj jezgara
- veličina problema
- tip skaliranja

program je izvršavan **30 puta**, a za svaku konfiguraciju su izračunati:

- srednje vreme izvršavanja
- standardna devijacija
- broj outlier-a ($|x - \mu| > 2\sigma$)

Ubrzanje je definisano kao:

$$S(n) = \frac{T_{seq}}{T_{par}(n)}$$

3. Jako skaliranje (Strong Scaling)

3.1 Opis eksperimenta

Kod jakog skaliranja veličina problema je konstantna.

Primer:

- matrica: 2000 × 2000
- 200 iteracija

Menja se samo broj procesa:

1, 2, 4, 8

3.2 Rezultati – Python

Threads	Par_mean	Speedup
1	11.42	0.69
2	6.27	1.27
4	6.00	1.32
8	4.27	1.86

Maksimalno ubrzanje:

$$S(8) \approx 1.86$$

Efikasnost:

$$E = \frac{S(n)}{n} = \frac{1.86}{8} \approx 23\%$$

3.3 Amdahl analiza - Python

Koristimo:

$$p = \frac{n(1 - \frac{1}{S(n)})}{n - 1}$$

Za n = 8 i S = 1.86 dobijamo:

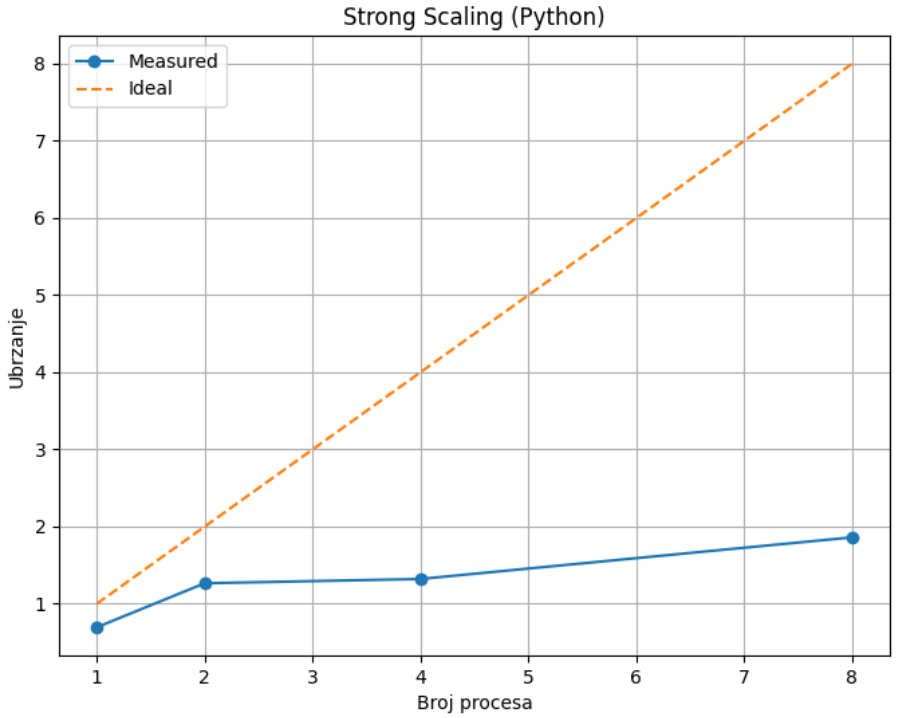
$$p \approx 0.69$$

To znači:

- Paralelni deo ≈ 69%
- Sekvencijalni deo ≈ 31%

Sekvencijalni deo uključuje:

- IPC komunikaciju
- deljenje blokova
- memorijski pristup
- multiprocessing overhead



3.4 Teorijski maksimum (Amdahl)

Ako je p = 0.69, teorijski maksimum (n → ∞):

$$S_{max} = \frac{1}{1-p} = \frac{1}{0.31} \approx 3.22$$

Dakle, Python implementacija nikada ne može preći ≈ 3.2x ubrzanje bez promene arhitekture.

3.5 Rezultati – Rust

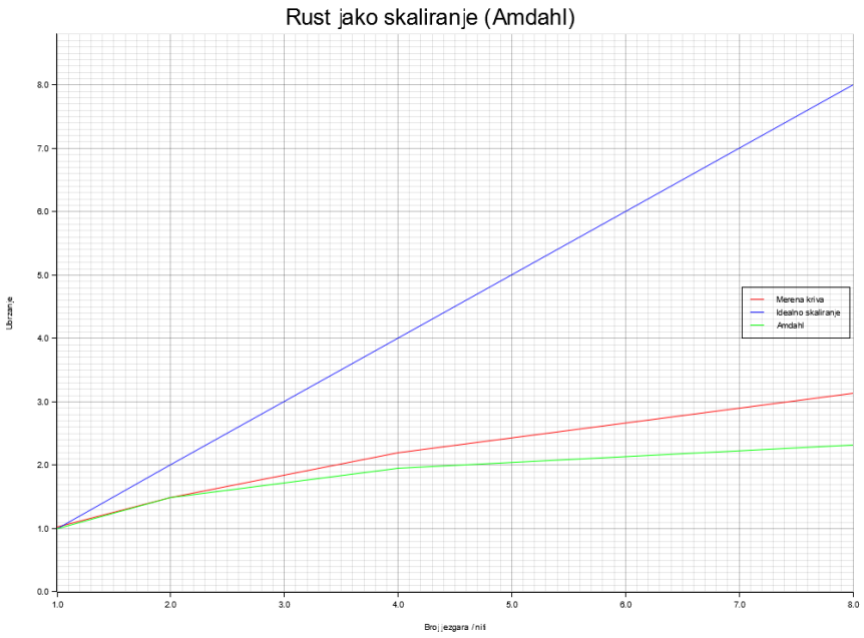
Eksperimentalni rezultati za Rust implementaciju (1000 × 1000, 200 iteracija):

Broj niti	Par_mean(s)	Ubrzanje
1	2.924	1.02
2	2.014	1.48
4	1.357	2.20
8	0.952	3.14

Maksimalno ostvareno ubrzanje:

$$S(8) \approx 3.14$$

Ovo predstavlja gotovo duplo bolje skaliranje u odnosu na Python implementaciju.



3.6 Procena paralelnog dela (Amdahl – Rust)

Koristeći Amdahl-ov zakon:

$$p = \frac{n\left(1 - \frac{1}{S(n)}\right)}{n - 1}$$

Za n = 8 i S = 3.14:

$$p \approx 0.84$$

To znači:

- Paralelni deo ≈ 84%
- Sekvencijalni deo ≈ 16%

U poređenju sa Python implementacijom (69% paralelizabilnog dela), Rust pokazuje znatno veći procenat paralelizabilnog koda.

3.7 Teorijski maksimum (Amdahl – Rust)

$$S_{max} = \frac{1}{1 - p}$$

$$S_{max} \approx \frac{1}{0.16} \approx 6.25$$

Dakle, teorijski maksimum za Rust implementaciju iznosi približno 6.25x, što je znatno više od Python maksimuma (≈ 3.2x).

4. Slabo skaliranje (Weak Scaling)

4.1 Opis eksperimenta

Kod slabog skaliranja veličina problema raste proporcionalno broju procesa.

Manipulacija poslom:

$$rows = base_{rows} * n$$

Time se održava konstantan broj redova po jezgru.

Na primer:

- 1 proces → 500 × 2000
- 2 procesa → 1000 × 2000
- 4 procesa → 2000 × 2000
- 8 procesa → 4000 × 2000

Time svaki proces obrađuje približno isti broj redova.

4.2 Rezultati - Python

Threads	Par_mean	Speedup
1	3.77	0.49
2	4.07	1.06
4	6.09	1.40
8	12.28	1.45

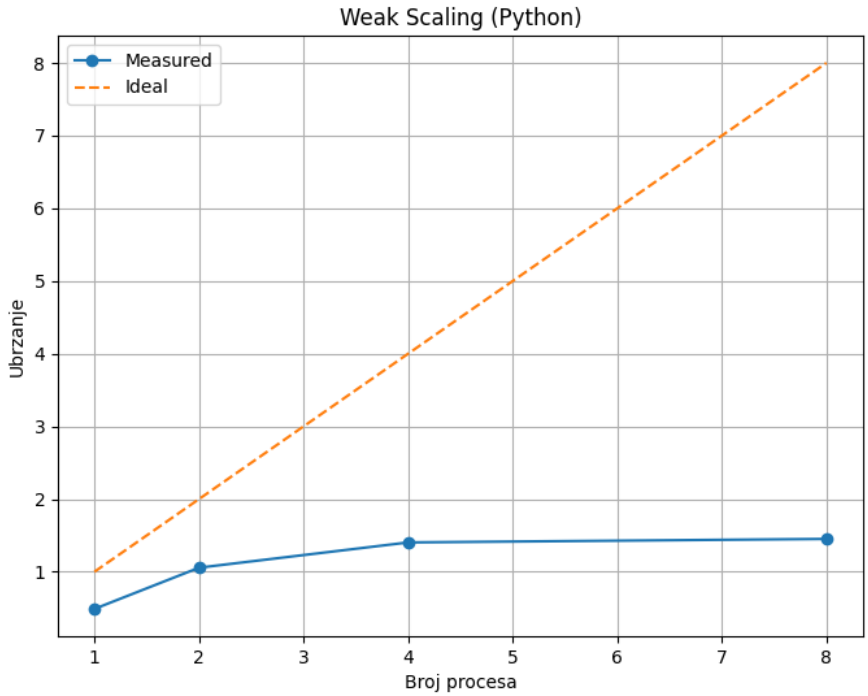
Maksimalno ubrzanje:

$$S(8) \approx 1.45$$

Weak scaling je slabiji od strong scaling-a kod Python-a u ovom merenju.

Razlog:

- veći overhead kod većih matrica
- Windows spawn
- memorijski pritisak



4.3 Rezultati - Rust

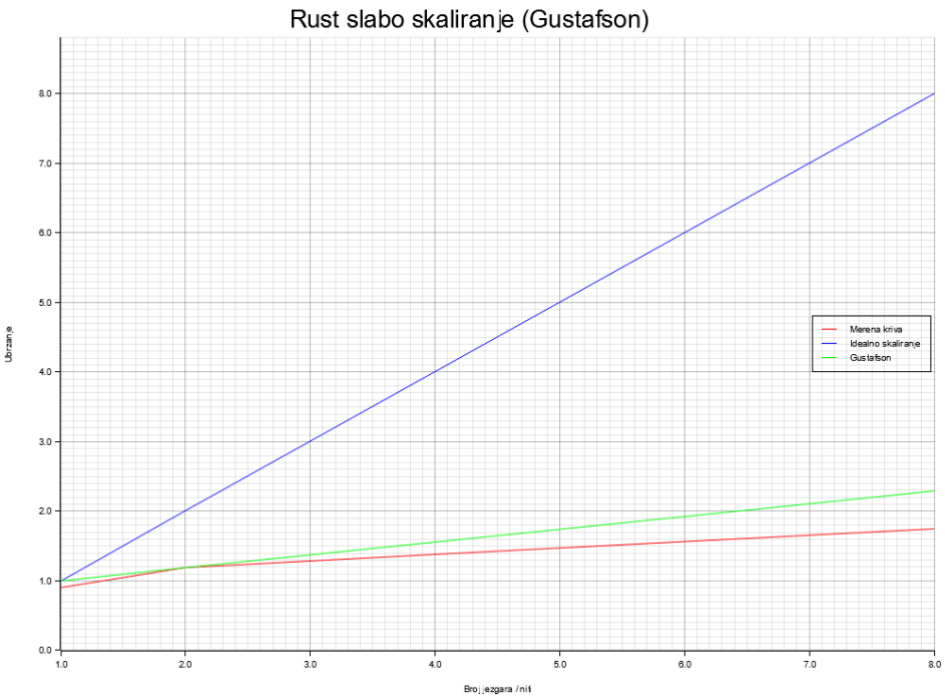
Rezultati slabog skaliranja (250 × 1000 po jezgru):

Broj niti	Par_mean(s)	Ubrzanje
1	0.695	0.90
2	0.430	1.18
4	0.373	1.37
8	0.294	1.75

Maksimalno ubrzanje:

$$S(8) \approx 1.75$$

Rezultati su stabilni i konzistentni sa Gustafson-ovim zakonom, ali je skaliranje ograničeno memorijskim protokom, budući da Game of Life predstavlja memory-bound problem.



4.4 Gustafson-ov zakon

$$S(n) = n - \alpha(n-1)$$

Iz rezultata se vidi da sekvencijalni deo i dalje ima značajan uticaj, ali paralelni deo dominira pri većim veličinama problema.

5. Poređenje Python i Rust implementacije

Osobina	Python	Rust
Strong scaling S(8)	1.86	3.14
Weak scaling	1.45	1.75
Paralelni deo p	$\approx 69\%$	$\approx 84\%$
Teorijski maksimum	≈ 3.2	≈ 6.25
Interpreter overhead	Prisutan	Ne postoji
IPC kopiranje	Da	Ne
Kontrola memorije	Ograničena (NumPy + multiprocessing)	Direktna i nativna
Efikasnost	~22%	Znatno veća

6. Uticaj hardverske arhitekture

Procesor Intel® Core™ Ultra 5 135U poseduje hibridnu arhitekturu sa kombinacijom Performance i Efficient jezgara. Takva organizacija može dovesti do neujednačenog skaliranja, jer različita jezgra imaju različite performanse.

Game of Life je izrazito memory-bound algoritam, što znači da je performansa dominantno ograničena:

- memorijskim protokom
- deljenim L3 cache-om
- pristupom glavnoj memoriji

Kako broj niti raste, dolazi do zagušenja memorijske magistrale, što ograničava linearno skaliranje.

7. Zaključak

Eksperimenti jakog i slabog skaliranja pokazali su značajne razlike između Python i Rust implementacije Conway-evog Game of Life algoritma.

U režimu jakog skaliranja, Python implementacija ostvaruje maksimalno ubrzanje od približno **1.86x** za 8 procesa. Na osnovu Amdahl-ovog zakona procenjen je paralelni deo od približno **69%**, dok sekvencijalni deo iznosi oko **31%**. Teorijski maksimum ubrzanja za Python implementaciju iznosi približno **3.2x**, što potvrđuje da je skaliranje ograničeno arhitekturom izvršavanja i multiprocessing overhead-om.

Rust implementacija ostvaruje znatno bolje rezultate. U režimu jakog skaliranja postignuto je maksimalno ubrzanje od približno **3.14x** za 8 niti. Procena paralelnog dela iznosi oko **84%**, dok sekvencijalni deo čini približno **16%** ukupnog vremena izvršavanja. Teorijski maksimum ubrzanja prema Amdahl-ovom zakonu iznosi približno **6.25x**, što ukazuje na znatno veći potencijal za paralelizaciju u odnosu na Python verziju.

U režimu slabog skaliranja, rezultati pokazuju da je skaliranje u oba jezika stabilnije nego kod jakog skaliranja, ali je i dalje ograničeno memorijskim protokom. Game of Life predstavlja izrazito **memory-bound problem**, pri čemu performanse dominantno zavise od brzine pristupa memoriji i deljenog keša, a ne od čiste računske snage procesora.

Rezultati jasno pokazuju da Rust implementacija omogućava značajno efikasniju paralelizaciju zahvaljujući:

- odsustvu interpreter overhead-a,
- direktnoj kontroli nad memorijom,
- izostanku međuprocesnog kopiranja podataka,
- nativnoj implementaciji paralelizacije.

Sa druge strane, Python multiprocessing pristup uvodi dodatni overhead (posebno na Windows platformi koja koristi spawn mehanizam), što značajno ograničava ostvarivo ubrzanje.

Zaključno, eksperimenti potvrđuju teorijske modele Amdahl-ovog i Gustafson-ovog zakona i jasno demonstriraju razliku između visokonivojske paralelizacije u interpretiranom jeziku i nativne sistemske paralelizacije u kompajliranom jeziku. Rust se pokazuje kao znatno pogodniji izbor za HPC scenarije i aplikacije koje zahtevaju visoko efikasnu paralelnu obradu.